

RAG / Retrieval System — Interview Questions

Reproduced exactly as originally written from Retrieval Augmented Generation (RAG)/RAG Workflow - Three Steps.md — nothing reworded.

Everything in the source file taught the concepts. This section rehearses them the way a real interview actually tests them — as back-and-forth dialogue, calibrated to what's expected from someone with roughly four years of AI/ML engineering experience: not just definitions, but trade-off reasoning, production awareness, and the ability to handle a follow-up push. Try answering each question out loud before reading the model answer.

What a 4-YOE candidate is expected to show, beyond definitions: - Correct terminology used naturally, not just recited on request - Trade-off reasoning ("X is faster but Y is more accurate, so in practice we do Z") - Production awareness — latency, cost, staleness, scale — not just textbook mechanics - Comfort saying "it depends," followed immediately by *what* it depends on

 **Interview Q1:** "Walk me through how the retrieval step of a RAG system works, end to end."

✓ **Strong answer:** "It splits into an offline and an online phase. Offline: documents get chunked, each chunk is run through an embedding model to produce a vector, and those vectors are stored and indexed in a vector database. Online: the user's query goes through that *same* embedding model, the database runs an approximate nearest-neighbor search — HNSW is the common choice — to find the closest stored vectors, and it returns the top-K matching chunks as plain text. Optionally, a cross-encoder reranks that shortlist for extra precision before the text gets handed to the generation step."

 **Interview Q2:** "What's the difference between dense and sparse retrieval, and when would you pick one over the other?"

✓ **Strong answer:** "Sparse retrieval — BM25, TF-IDF — matches on exact words; it's a huge, mostly-zero vector with one slot per vocabulary word. Dense retrieval uses embeddings — short vectors where every value is meaningful — and matches on semantic similarity, so it finds 'recover your login credentials' when someone asks to 'reset my password,' which sparse search would miss entirely. The trade-off: sparse still wins on exact identifiers — order numbers, error codes, legal citations — where a 'close' match is worthless. In practice, most production systems run both together as hybrid search rather than picking one."

 **Interview Q2 (follow-up):** "Say I'm building search for a legal case database, full of citations and case numbers. Which would you lean toward?"

✓ **Strong answer:** "I'd lean hybrid, but weighted toward sparse for anything that looks like an identifier. Case numbers and citations need exact matching — dense retrieval could easily return a 'semantically similar' but wrong case. I'd probably route obviously-structured queries (a citation pattern) through keyword search directly, and free-text legal questions through dense retrieval, merging results when a query has both."

 **Interview Q3:** "Why must the same embedding model be used for both the documents and the query?"

✓ **Strong answer:** "Each embedding model defines its own vector space — its own private 'coordinate system' for meaning. Two different models can't be compared, the same way a rating out of 10 and a rating out of 100 aren't comparable side by side even if they represent the same enthusiasm. Practically, this means an embedding-model upgrade isn't a config change — it requires re-embedding and re-indexing the entire corpus, which is a real operational cost to plan for."

 **Interview Q4:** "Why not just do exact nearest-neighbor search? What does an index like HNSW actually buy you?"

✅ **Strong answer, broken down simply:** "Exact search means checking the query against **every single stored vector**, one by one. If you have 10 million documents, that's 10 million comparisons for every question — fine for a small dataset, too slow once it gets large. That's the problem.

HNSW solves it by pre-building a kind of shortcut map, ahead of time, so you never have to check everything. Picture a highway system: a few big 'highway' connections let you jump straight into roughly the right neighborhood fast, and then smaller 'local road' connections let you fine-tune from there to the actual closest matches. So instead of checking every single document, the search just hops through a handful of these pre-built connections — a handful of steps instead of millions.

This is called **Approximate Nearest Neighbor search, or ANN** — 'approximate' because you might, very occasionally, miss the single best match by a tiny margin. In exchange, you get a massive speed boost, which is a trade that's almost always worth making.

At an even bigger scale — hundreds of millions of vectors — you'd also shrink each vector itself using a technique called **quantization**. Here's what that means concretely, with an example:

```
BEFORE quantization – full precision, 4 bytes per number:
[0.82719348, -0.31402213, 0.55098721, 0.12345678, ...]
→ 768 numbers × 4 bytes ≈ 3,072 bytes (~3 KB) for ONE vector

AFTER quantization – compressed, ~1 byte per number:
[0.83, -0.31, 0.55, 0.12, ...]
→ 768 numbers × 1 byte ≈ 768 bytes (~0.75 KB) for the SAME vector
→ roughly 75% less memory, for that one vector
```

Each individual number gets rounded to something coarser — '0.83' instead of '0.82719348' — so it needs far fewer bits to store. Multiply that saving across 100 million vectors and the difference is enormous: roughly 300 GB of raw vectors shrinks to well under 100 GB. You do lose a tiny bit of precision on every number, but the vector still lands almost exactly where it did before in 'meaning space,' so the search still finds essentially the same closest matches — which is exactly the photo-compression analogy: compressing a photo throws away detail too fine to notice, while the picture still looks basically identical."

 **Interview Q5:** "How would you decide on a chunking strategy for a new document set?"

✅ **Strong answer:** "I'd start with the failure modes chunking causes: chunks too small lose surrounding context; chunks too large dilute relevance and burn through the context window budget. Common strategies, roughly in order of sophistication: fixed-size chunking as a quick baseline; recursive/character-aware splitting that respects paragraph and sentence boundaries instead of cutting mid-sentence; semantic chunking, which splits where the *topic* actually shifts; and hierarchical parent-child chunking — retrieve on small, precise child chunks, but return the larger parent chunk to the LLM so it has full context. I'd also add a small overlap between consecutive chunks so relevant information sitting right at a chunk boundary doesn't get orphaned."

 **Interview Q6:** "How do you choose top-K, and where does reranking fit in?"

✓ **Strong answer:** "Too small a K risks missing the right chunk entirely; too large wastes context window budget and can dilute the model's attention across irrelevant text. The common production pattern is to decouple the two: retrieve a wider net cheaply — say the top 50 — with the bi-encoder and ANN search, then use a cross-encoder to rerank that shortlist down to a much smaller K, maybe 5, before it goes to the LLM. That gets you both the speed of ANN and the precision of a slower, pairwise model, without ever running the expensive model over the whole corpus."

🎙 **Interview Q7:** "How would you evaluate whether your retrieval step is actually working well?"

✓ **Strong answer, rebuilt at the simplest level, with a plain analogy for each metric:**

Step 0 — the thing all four metrics quietly assume: none of these four numbers can be computed unless you *already know the correct answer ahead of time*. That known answer key is called **ground truth** (or a **labeled evaluation set**) — a prepared list of (query → which documents are actually correct for it), usually built by a human expert judging relevance, or from historical data like which document users actually clicked on. This is exactly right to flag: you cannot compute Precision, Recall, MRR, or NDCG for a brand-new live query where nobody knows the right answer — these are **offline tests**, run against a prepared answer key you built in advance, specifically so you can measure quality before real users ever see the system.

One simple analogy underneath all four — fishing in a lake where you already know exactly which fish you want:

Say you already know (ground truth) there are **3 target fish** hiding somewhere in the lake. You cast a net and pull up **5 fish total**, and **2 of the 5** happen to be target fish.

- **Precision — 'Of what I pulled up, how much was actually what I wanted?'** You pulled up 5 fish, 2 were targets → **Precision = $2/5 = 40\%$** . This is about the *purity of your catch* — it doesn't care how many target fish exist, only how much of what's in your net is actually good.
- **Recall — 'Of everything I was hoping to catch, how much did I actually get?' (you had this exactly right)** There were 3 target fish total in the lake, you caught 2 of them → **Recall = $2/3 \approx 67\%$** . This is about *completeness* — it doesn't care how much junk is also in your net, only how much of the good stuff you didn't miss.
- **MRR, short for Mean Reciprocal Rank — 'How many empty nets did I pull up before I got my FIRST target fish?'** Forget the rest of the catch entirely — MRR only asks: was your very first fish a target? If yes, perfect score. If your first 2 pulls were junk and the 3rd was finally a target fish, that's a worse score, even if your final net eventually had plenty of good fish in it. It's purely about **how fast you hit something good**, averaged across many separate fishing trips (queries).
- **NDCG, short for Normalized Discounted Cumulative Gain — 'Not just did I catch good fish — did I catch the BEST fish first?'** Say one of your target fish is a huge prize catch, and the other is a small, so-so one. NDCG checks whether the prize fish came up *before* the so-so one. If your net order was [prize fish, junk, junk, so-so fish], that scores well. If it was [so-so fish, junk, junk, prize fish] — same 2 fish caught, same precision and recall — NDCG scores it *worse*, because your best catch was buried near the bottom instead of sitting at the top.

Mapped onto the actual example (query: 'reset password,' ground truth: A, D, F are correct; system returned A, B, C, D, E in that order):

Precision@5 = 2/5 = 40% (A and D, out of the 5 shown, are correct)
Recall@5 = 2/3 ≈ 67% (A and D, out of the 3 that truly exist, were found)
MRR = 1/1 = 1 (the very first result, A, was already correct)
NDCG = high (assuming A is the strongest answer, it's sitting at #1 — exactly where it should be)

F, the one truly relevant doc that never showed up at all, is exactly what Recall is built to catch — Precision and MRR wouldn't even notice it's missing."

Metric	The one-line question it answers
Precision@K	Of what I showed, how much was actually good?
Recall@K	Of everything good that exists, how much did I find?
MRR (Mean Reciprocal Rank)	How fast did I hit the first good one?
NDCG (Normalized Discounted Cumulative Gain)	Is my <i>very best</i> result sitting at the top, not buried?

 **Interview Q7 (follow-up 1):** "Two things — since retrieval isn't random like generation, would asking the exact same query multiple times ever change the MRR? And if my average MRR across ten test queries comes out to something like 0.55, does that tell me anything about which specific query is dragging the score down?"

 **Strong answer:** "On the first part — no, it shouldn't. Retrieval is deterministic for a fixed query against an unchanged index: same embedding model, same vector, same ANN search, same result, every time. If I actually saw a repeated query's MRR change, I'd suspect something upstream — the index got updated between calls, or there's a query-rewriting step feeding in some randomness before the embedding happens.

On the second part — no, the average alone can't tell you that, and it's a real trap. An average MRR of 0.55 across ten queries could mean every single query scored right around 0.55. But it could just as easily mean five queries scored a perfect 1.0 and five scored 0.1 — both cases average out to exactly the same 0.55. Those are two very different systems: one consistently mediocre, the other great half the time and badly broken the other half. To actually find the weak query, you need the per-query reciprocal rank, not just the aggregate — sort them worst to best and look for what the worst performers have in common."

 **Interview Q7 (follow-up 2):** "Correct me if I'm wrong — if MRR for a particular query is 1, that means NDCG is good too, right? And maybe vice versa?"

 **Strong answer:** "That's a really natural assumption, but it's not guaranteed in either direction — they're measuring genuinely different things. MRR only checks *whether something relevant showed up at position 1* — it doesn't care whether that something was your *best* content or just barely relevant, and it completely ignores everything below position 1. NDCG cares about the whole list, weighted by *degree* of relevance, so it can absolutely catch a problem that MRR is blind to.

Here's a concrete case where MRR is perfect but NDCG isn't. Say for one query there are two genuinely relevant documents: X, a perfect answer, and Y, only barely relevant. Compare two possible rankings:

Ranking A: [Y (barely relevant), X (perfect), Z (irrelevant)]
→ MRR: first relevant doc is Y, at position 1 → $MRR = 1/1 = 1$ (perfect)
→ NDCG: the BEST doc, X, is buried at position 2 instead of 1
→ this isn't the ideal order → $NDCG < 1$ (imperfect)

Ranking B: [X (perfect), Y (barely relevant), Z (irrelevant)]
→ MRR: first relevant doc is X, at position 1 → $MRR = 1/1 = 1$ (perfect)
→ NDCG: this IS the ideal order → $NDCG = 1$ (perfect)

Ranking A and Ranking B score an **identical, perfect MRR of 1** — MRR genuinely cannot tell them apart. But Ranking B is clearly the better system: it put its strongest answer first. NDCG correctly scores B higher than A, because NDCG asks 'is my *best* content at the top,' not just 'is *something* relevant at the top.'

For the 'vice versa' half — a genuinely high NDCG usually does correlate with a decent MRR, since a near-ideal overall ordering tends to have a good item at the top too. But it's not a strict guarantee, especially in a longer results list: NDCG gives credit for relevant content anywhere in the list, discounted by position, so it's possible to have a reasonably good NDCG even when the very top slot isn't perfectly optimal, as long as the rest of the ranking is strong. MRR, by contrast, only ever looks at that first position and nothing else.

So the honest summary: they usually move together in the simple case — one relevant document, or all relevant documents roughly equally good — but they diverge exactly when there are multiple relevant documents of *different* quality and the best one isn't sitting exactly on top. That's precisely the situation MRR is blind to and NDCG is built to catch."

 **Interview Q7 (follow-up 3):** "So for Ranking A and Ranking B, you gave $MRR = 1$ for both. But shouldn't we instead measure MRR against the already-known perfect document specifically? In Ranking A, the perfect document X is at position 2, so shouldn't MRR be $1/2 = 0.5$, not 1?"

 **Strong answer:** "Your math is exactly right for what you're describing — X sitting at position 2 does give $1/2 = 0.5$ under that definition. But here's the important part: what you just described isn't how *standard* MRR is defined — and this is actually the exact reasoning that led to NDCG being invented as a separate metric.

Classic MRR uses **binary relevance**: a document is either 'relevant' or 'not relevant,' full stop — there's no built-in concept of *which relevant document is the best one*. The reciprocal rank is $1 / (\text{position of the first document marked relevant, at any grade})$. That's precisely why Ranking A scored $MRR = 1$ in the earlier example: Y (barely relevant) and X (perfect) were both simply 'relevant' as far as MRR's binary check is concerned — MRR has no way to say 'relevant, but not the *best* one.'

If instead you specifically tracked 'where does the single known-best document rank' — which is what you're proposing — that's a real, meaningful thing to measure, and your calculation of 0.5 for it is correct. But that specific idea, 'don't just check if something relevant is on top, check if the *best known* thing is on top,' is exactly the philosophy NDCG was built around — except NDCG does it more completely, checking the position of *every* relevant document, weighted by its grade, across the *whole* list, not just the single best one.

	What it checks	Ranking A's score
Standard MRR	Position of the <i>first</i> item marked relevant, any grade	$1/1 = 1$
Your proposed version	Position of the single <i>known-best</i> document	$1/2 = 0.5$
NDCG	Position of <i>every</i> relevant document, weighted by grade, across the whole list	less than 1

Your version and NDCG are catching the exact same problem — X isn't where it should be — NDCG is just the fuller, standardized version of that same instinct, extended to the entire ranked list."

 **Interview Q8:** "How would you scale this to a billion vectors, and what breaks first?"

✓ **Strong answer:** "Memory usually breaks first — HNSW graphs are memory-hungry, so at that scale you'd look at quantization to shrink each vector's footprint, or a disk-backed ANN index accepting some latency for lower cost. You'd also shard across multiple nodes and query them in parallel. The other thing that breaks is index freshness: rebuilding a billion-vector index from scratch isn't instant, so you need an incremental upsert strategy for new or changed documents, and a re-embedding plan for whenever the embedding model itself changes."

 **Interview Q9:** "Your RAG system is giving wrong answers. How do you figure out whether it's a retrieval problem or a generation problem?"

✓ **Strong answer:** "Split the pipeline apart and inspect it at the seam. Log exactly which chunks were retrieved for the failing query, and check by hand: is the correct information even in the top-K? If it's *not* in there, that's a retrieval problem — look at chunking, the embedding model, or top-K. If the right chunk *is* in there and the model still got it wrong, that's a generation problem — check prompt construction, whether the context window overflowed, or the model's own reasoning. Debugging RAG without separating these two steps just leads to guessing."

 **Interview Q10:** "If I told you to make retrieval more accurate without touching the embedding model at all, what would you try?"

✓ **Strong answer:** "Several levers don't touch the embedding model: improve the chunking strategy and add overlap; add a cross-encoder reranking stage on top of the existing retrieval; add hybrid search so exact-match cases dense retrieval misses still get caught by keyword search; widen top-K before reranking to reduce the chance of missing the right chunk in the first pass; add metadata filtering — like filtering by date or category before the vector search — to shrink the search space; and query rewriting, where you use the LLM itself to expand or clarify a vague user query into a better search query before it's ever embedded."

What interviewers are really scoring for, across all of the above: - Whether you reach for the right *term* naturally (ANN, HNSW, bi-encoder, cross-encoder, top-K) instead of describing around it - Whether you can articulate a trade-off instead of declaring one option universally "better" - Whether "how would you evaluate this" gets a concrete answer (metrics, a labeled eval set) instead of "check if it looks right" - Whether a debugging question gets a *method* (isolate retrieval from generation) rather than a guess - Whether a constrained/curveball question ("without touching X") produces multiple concrete levers, not a shrug

Sources consulted while calibrating this section: - [Top 30 RAG Interview Questions and Answers for 2026 — DataCamp](#) - [RAG Interview System — 548 questions & system design scenarios \(GitHub\)](#) - [Top Interview Questions on RAG for Data Science and AI Engineer Roles](#) - [RAG & Vector Database Interview Questions for 2026 — TopGenAIJobs](#)